

```

int main()
{
    BIO      *out=NULL, *ret=NULL, *con=NULL, *ssl_bio=NULL;
    SSL_CTX  *ctx = NULL;
    //SSL     *ssl = NULL;
    int      len = 0;
    FILE     *fd = NULL;
    char     tmpbuf[1024];

    /* Initialize */
    1_InitOpenSSL();
    RET(ctx = SSL_CTX_new(TLSv1_client_method()));
    /* For Client authentication */
    if( 1 == 1_SetPrivateKeyAndCert(ctx) ) {
        printf(" key init failed \n");
    }
    /* ===== */
    RET(con=BIO_new(&methods_connect));
    RET(ssl_bio=BIO_new_ssl(ctx,1));
    RET(ret=BIO_push(ssl_bio,con));
    BIO_get_ssl(ssl_bio, &ssl);
    if (!ssl) {
        fprintf(stderr, "Can't locate SSL pointer\n");
        goto EXIT;
    }
    SSL_set_mode(ssl, SSL_MODE_AUTO_RETRY);
    RET(fd = fopen(OUT_FILE, "w"));
    RET(out = BIO_new_fp(fd, BIO_NOCLOSE));
    if (BIO_do_connect(ssl_bio) <= 0) {
        fprintf(stderr, "Error connecting to server\n");
        printf("state = %s \n",SSL_state_string_long(ssl));
        ERR_print_errors_fp(stderr);
        goto EXIT;
    }

    BIO_puts(ssl_bio, "GET <some specific file> HTTP/1.1\nHost:
<IP address>:443\n\n");
    for (;;) {
        len = BIO_read(ssl_bio, tmpbuf, 1024);
        if (len <= 0) {
            break;
        }
        BIO_write(out, tmpbuf, len);
    }
    SSL_shutdown(ssl);
    printf("state read = %s \n",SSL_state_string_
long(ssl));
EXIT:
    BIO_free_all(ssl_bio);
    BIO_free(out);
    fclose(fd);
    return 0;
}

```

Finally, customise only if you have to

Everything has a flip side to it. For example, hardware acceleration is a good thing, but it comes with its own drawbacks like initial costs, the cost of upgrading, defect fixing, etc. There may be easier alternatives already supported in OpenSSL. One such example is the AES-NI support.

In order to achieve faster speeds, the x86 instruction set was extended to include the Advanced Encryption Standard New Instruction (AES-NI) set of instructions that are supported on some Intel and AMD processors as detailed in http://en.wikipedia.org/wiki/AES_instruction_set.

From version 1.0.1 onwards, OpenSSL supports the use of the AES-NI instruction set in order to speed up AES operations. **END** 

References

- [1] Accelerating OpenSSL using Intel QuickAssist technology: <http://www.intel.com/content/dam/www/public/us/en/documents/solution-briefs/accelerating-openssl-brief.pdf>
- [2] Global Internet Phenomena Report: [www.electronics.dti.ie/staff/dclarke/Other Files/Sandvine_Global_Internet_Phenomena_Report_2H_2012.pdf](http://www.electronics.dti.ie/staff/dclarke/Other%20Files/Sandvine_Global_Internet_Phenomena_Report_2H_2012.pdf)
- [3] Wikipedia on OpenSSL: <http://en.wikipedia.org/wiki/OpenSSL>

By: Ravi Honnavalli

The author is a software professional working extensively on open source software for network and embedded systems security.



PACK WEB HOSTING
Time to go PRO now

 www.packwebhosting.com
 **0-98769-44977**
 support@packwebhosting.com

A Leading Web & Email Hosting Provider

Specialists in

Hosting Sites

built with OpenSource Technologies


WordPress


Magento


Joomla


Drupal

ProX Plans
Have a High Traffic Website?
Considering VPS/Server?
Visit prox.packwebhosting.com

- cPanel Hosting
- One Click Installation
- Solid Support
- Multiple Hosting Plans

Why Us?

- 4000+ Hosting
- 6500+ Domains
- 2000+ Clients
- 11+ Years Experience

Trust Us. Trust our Ability.